

C 语言内存管理编程题 · 参考答案

对应《C 语言内存管理编程题》题一 ~ 题三 · 代码均以 C11 编写，可配合 ASan/Valgrind 直接运行

题一 参考答案

核心函数

```
#include <stdlib.h>
#include <stddef.h>

char *my_strdup(const char *s) {
    if (!s) return NULL;
    size_t n = 0;
    while (s[n]) n++;          /* 自己数长度，不用 strlen 也行 */
    char *p = malloc(n + 1);
    if (!p) return NULL;
    for (size_t i = 0; i <= n; i++) p[i] = s[i];
    return p;
}

char *my_strndup(const char *s, size_t n) {
    if (!s) return NULL;
    size_t len = 0;
    while (len < n && s[len]) len++; /* 取 min(strlen(s), n) */
    char *p = malloc(len + 1);
    if (!p) return NULL;
    for (size_t i = 0; i < len; i++) p[i] = s[i];
    p[len] = '\0';
    return p;
}
```

完整主程序（含不定长行读入）

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

/* 读入一整行（不含换行），行长度不限；失败返回 NULL */
static char *read_line(void) {
    size_t cap = 16, len = 0;
    char *buf = malloc(cap);
    if (!buf) return NULL;

    int c;
    while ((c = getchar()) != EOF && c != '\n') {
        if (len + 1 >= cap) {          /* 预留 '\0' 的位置 */
            if (cap > SIZE_MAX / 2) { free(buf); return NULL; }
            size_t ncap = cap * 2;
            char *tmp = realloc(buf, ncap);
            if (!tmp) { free(buf); return NULL; } /* 失败不丢原指针 */
            buf = tmp; cap = ncap;
        }
        buf[len++] = c;
    }
    buf[len] = '\0';
    return buf;
}
```

```

    }
    buf[len++] = (char)c;
}
buf[len] = '\0';
return buf;
}

int main(void) {
    int n;
    if (scanf("%d", &n) != 1 || n < 0) return 1;
    while (getchar() != '\n') { } /* 吃掉第一行剩余字符 */

    char **lines = malloc(n ? (size_t)n * sizeof(char *) : 1);
    if (!lines) { puts("alloc failed"); return 1; }

    for (int i = 0; i < n; i++) {
        lines[i] = read_line();
        if (!lines[i]) { /* 失败：回收已成功的部分 */
            for (int j = 0; j < i; j++) free(lines[j]);
            free(lines);
            puts("alloc failed");
            return 1;
        }
    }

    for (int i = n - 1; i >= 0; i--) { /* 逆序输出并逐行释放 */
        puts(lines[i]);
        free(lines[i]);
    }
    free(lines);
    return 0;
}

```

评分要点：

1. 两个复制函数正确处理 `NULL`、空串与 `n = 0`。
2. `read_line` 支持含空格的空行（不能只用 `scanf("%s")`）；`realloc` 用临时指针接返回值。
3. 失败路径把已分配的行逐个 `free` 再退出，ASan 无泄漏报告。
4. `N = 0` 时仍然 `free(NULL)` 安全（或按上面的写法分配 1 字节，二者皆可）。

常见错误：用 `char buf[1001]` 读行（题目要求每行单独堆分配）；`buf[len] = '\0'` 前没检查 `len + 1 >= cap` 导致堆溢出；只 `free(lines)` 忘记逐行释放造成泄漏。

题二 参考答案

```
#include <stdlib.h>
#include <stdint.h>

int vec_init(IntVec *v) {
    v->data = NULL;
    v->size = v->capacity = 0;
    return 0;
}

int vec_reserve(IntVec *v, size_t need) {
    if (need <= v->capacity) return 0;
    if (need > SIZE_MAX / sizeof(int)) return -1; /* ① 先查整型溢出 */

    size_t cap = v->capacity ? v->capacity : 1;
    while (cap < need) {
        if (cap > SIZE_MAX / 2) { cap = need; break; } /* ② 倍增也会溢出 */
        cap *= 2;
    }

    int *tmp = realloc(v->data, cap * sizeof(int)); /* ③ 用临时变量接 */
    if (!tmp) return -1; /* 失败时 v 原样不动 */
    v->data = tmp;
    v->capacity = cap;
    return 0;
}

int vec_push(IntVec *v, int x) {
    if (vec_reserve(v, v->size + 1) != 0) return -1;
    v->data[v->size++] = x;
    return 0;
}

void vec_clear(IntVec *v) { v->size = 0; }

void vec_free(IntVec *v) {
    free(v->data);
    v->data = NULL;
    v->size = v->capacity = 0;
}

/* 加分项 */
int vec_shrink_to_fit(IntVec *v) {
    if (v->size == v->capacity) return 0;
    if (v->size == 0) { /* 缩到 0: 直接释放 */
        free(v->data);
        v->data = NULL; v->capacity = 0;
        return 0;
    }
    int *tmp = realloc(v->data, v->size * sizeof(int));
    if (!tmp) return -1; /* 缩容失败不算错, 但保持原状 */
    v->data = tmp;
    v->capacity = v->size;
}
```

```
    return 0;
}
```

主程序

```
#include <stdio.h>

static int cmp_int(const void *a, const void *b) {
    int x = *(const int *)a, y = *(const int *)b;
    return (x > y) - (x < y);          /* 避免相减溢出 */
}

int main(void) {
    size_t n;
    if (scanf("%zu", &n) != 1) return 1;

    IntVec v;
    vec_init(&v);
    for (size_t i = 0; i < n; i++) {
        int x;
        if (scanf("%d", &x) != 1) { vec_free(&v); return 1; }
        if (vec_push(&v, x) != 0) { vec_free(&v); puts("alloc failed"); return 1; }
    }

    qsort(v.data, v.size, sizeof(int), cmp_int);
    for (size_t i = 0; i < v.size; i++)
        printf("%d%c", v.data[i], i + 1 == v.size ? '\n' : ' ');
    if (v.size == 0) putchar('\n');

    vec_free(&v);
    return 0;
}
```

均摊分析：容量按 1, 2, 4, ..., n 增长，插入 n 个元素的总搬迁代价为 $1 + 2 + 4 + \dots + n < 2n$ ，故每次 `vec_push` 均摊 $O(1)$ ；若每次只 +1 则退化为 $O(n^2)$ 。

评分要点：失败时 `v` 状态不变（“强异常安全”）、溢出检查、`realloc` 临时指针、`vec_free` 可重复调用。

题三 参考答案

核心实现 (first-fit + split + coalesce)

```
#include <stddef.h>
#include <stdint.h>

#define ARENA_SIZE (1u << 20)

typedef struct {
    size_t size;    /* 本块总大小（含块头），恒为 8 的倍数 */
    int    free;    /* 1 = 空闲, 0 = 已分配 */
} Block;

#define HDR          ((sizeof(Block) + 7) & ~(size_t)7)    /* 64 位下为 16 */
#define ARENA_END (arena + ARENA_SIZE)

static unsigned char arena[ARENA_SIZE] __attribute__((aligned(16)));
static Block *head;

static size_t align8(size_t n) { return (n + 7) & ~(size_t)7; }

/* 下一块的地址；已经越过末尾则返回 NULL（遍历终止条件） */
static Block *blk_next(Block *b) {
    unsigned char *p = (unsigned char *)b + b->size;
    return (p >= ARENA_END) ? NULL : (Block *)p;
}

void my_init(void) {
    head = (Block *)arena;
    head->size = ARENA_SIZE;
    head->free = 1;
}

void *my_malloc(size_t size) {
    if (size == 0) size = 1;                                /* 约定：0 按 1 字节处理 */

    size_t need = align8(size);
    if (need < size || need > SIZE_MAX - HDR) return NULL; /* 对齐/相加溢出 */
    need += HDR;

    for (Block *b = head; b; b = blk_next(b)) {
        if (!b->free || b->size < need) continue;
        if (b->size - need >= HDR + 8) {                    /* 剩余够放块头 + 8B 才分裂 */
            Block *nb = (Block *)((unsigned char *)b + need);
            nb->size = b->size - need;
            nb->free = 1;
            b->size = need;
        }
        b->free = 0;
        return (unsigned char *)b + HDR;
    }
    return NULL;
}
```

```

/* 自头到尾扫一遍，把相邻空闲块合并 */
static void coalesce(void) {
    Block *p = head;
    while (p) {
        Block *n = blk_next(p);
        if (n && p->free && n->free) { p->size += n->size; continue; }
        p = n;
    }
}

void my_free(void *ptr) {
    if (!ptr) return;
    Block *b = (Block *)((unsigned char *)ptr - HDR);
    b->free = 1;
    coalesce();
}

size_t my_usable_size(void *ptr) {
    if (!ptr) return 0;
    return ((Block *)((unsigned char *)ptr - HDR))->size - HDR;
}

size_t my_free_block_count(void) {
    size_t c = 0;
    for (Block *b = head; b; b = blk_next(b)) if (b->free) c++;
    return c;
}

```

关键设计说明

- **对齐**: arena 按 16 字节对齐, HDR = 16, 且所有块大小都向上取整到 8 的倍数, 因此用户指针必然 8 字节对齐。
- **块大小语义**: size 含块头, 不含“分裂后残留块头”之外的空隙; my_usable_size = size - HDR。
- **终止条件**: blk_next 用 p >= ARENA_END 判断, 不会越界读写, 也不会死循环。
- **分裂阈值**: 残留空间 < HDR + 8 时不予分裂, 避免产生无法再利用的碎块 (内部碎片可控)。
- **复杂度**: 本实现每次 my_free 全表扫描, O(n)。若要 O(1), 可改用**边界标记 (boundary tag)**在物理相邻处直接找到前后块, 或用**显式空闲链表**只遍历空闲块。

测试驱动 (随机 10^5 次 + canary + 重叠检查)

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <assert.h>

#define MAXLIVE 4096
#define CANARY 0xA5

static void *live[MAXLIVE];
static size_t lsize[MAXLIVE];
static size_t nl;

static int overlaps(void *p, size_t n) {
    uintptr_t a = (uintptr_t)p, b = a + n;

```

```

    for (size_t i = 0; i < nl; i++) {
        uintptr_t c = (uintptr_t)live[i], d = c + lsize[i];
        if (a < d && c < b) return 1;
    }
    return 0;
}

int main(void) {
    my_init();
    srand(12345);

    for (int step = 0; step < 100000; step++) {
        int do_alloc = (nl == 0) || (nl < MAXLIVE && rand() % 2);
        if (do_alloc) {
            size_t n = (size_t)(rand() % 4096) + 1;
            char *p = my_malloc(n);
            if (!p) continue; /* 内存不足是合法结果 */
            assert(my_usable_size(p) >= n);
            assert(((uintptr_t)p % 8) == 0); /* 8 字节对齐 */
            assert(!overlaps(p, n)); /* 互不重叠 */
            assert(p >= (char *)arena && p + n <= (char *)arena + ARENA_SIZE);
            memset(p, CANARY, n);
            live[nl] = p; lsize[nl] = n; nl++;
        } else {
            size_t i = (size_t)(rand() % nl);
            char *p = live[i];
            for (size_t k = 0; k < lsize[i]; k++)
                assert((unsigned char)p[k] == CANARY); /* 用户区没被踩坏 */
            my_free(p);
            live[i] = live[--nl]; lsize[i] = lsize[nl]; /* swap-remove */
        }
    }

    while (nl) my_free(live[--nl]);
    printf("free blocks after all free = %zu\n", my_free_block_count());
    assert(my_free_block_count() == 1); /* 全部合并回一整块 */
    puts("all tests passed");
    return 0;
}

```

评分要点

1. 分配器只用静态数组：出现 `malloc/free` 调用即本题 0 分。
2. 对齐、溢出保护、`my_malloc(0)` 的注释约定。
3. 释放时必须合并相邻空闲块；全部释放后空闲块数为 1 是强判据。
4. 测试驱动能真正触发重叠/canary 断言，而不是只跑随机函数。

常见错误：块头只记 payload 大小导致无法定位下一块；分裂后忘记更新原块 `size`；`my_free` 只置 `free = 1` 不合并，运行几千次后 `my_malloc(64)` 就开始失败；用 `int` 保存块大小，在 1 MiB 区间虽然侥幸可用，但块数增多后与 `size_t` 混用出隐患。